

1. System Overview

The system is designed for fully CPU-based inference, with no dependency on GPU hardware at any stage of the pipeline.

The corpus consists of Sanskrit literary texts in .docx format, containing the following stories and passages:

Total corpus size: ~9,064 characters
Total chunks produced: 26 chunks (at 500 char chunk size, 100 char overlap)

3.1 Document Loading

3.2 Text Cleaning

- Removal of zero-width Unicode characters (U+200B, U+200C, U+200D, U+FEFF) common in copy-pasted Devanagari text
- Normalization of excessive whitespace and blank lines
- No transliteration normalization (Devanagari and Latin forms are treated as distinct tokens)

Design decision: No Sanskrit-specific NLP preprocessing (stemming, sandhi splitting, morphological analysis) was applied. This was a deliberate choice: the corpus mixes Devanagari with English transliteration in ways that Sanskrit NLP toolkits do not handle reliably. Treating all tokens as opaque Unicode strings produces more predictable retrieval behavior.

3.3 Chunking

- **Target chunk size:** 500 characters
- **Overlap:** 100 characters between consecutive chunks
- **Splitting priority:** Paragraph boundaries → sentence-like boundaries → hard character split
- **Rationale:** Paragraph boundaries in Sanskrit literary text generally align with narrative units (sentences, shloka verses, or dialogue exchanges), making them natural retrieval units.

4.1 Approach: TF-IDF with Cosine Similarity

A TF-IDF (Term Frequency–Inverse Document Frequency) retriever was implemented using only `numpy` and Python standard library – no external ML framework required.

Why TF-IDF for Sanskrit?

Dense vector embedding models (e.g., `sentence-transformers`) require training data in the target language to produce semantically meaningful representations. No widely available small embedding model has been trained on Sanskrit text. Applying a multilingual embedding model would project Sanskrit tokens into a semantic space calibrated on other languages, producing unreliable similarity scores.

TF-IDF, by contrast, operates purely at the token frequency level. It assigns higher weights to tokens that are distinctive within a document relative to the full corpus – a language-agnostic property that holds equally for Devanagari and Latin Unicode tokens.

4.2 Implementation Details

Tokenization: Regex-based extraction of Devanagari Unicode ranges (`\u0900-\u097F`) and ASCII Latin words. Latin tokens are lowercased; Devanagari tokens are preserved as-is.

TF computation: Term count normalized by total document token count (relative TF).

IDF computation: Smoothed IDF: $\log((N+1)/(df+1)) + 1$, where N = number of chunks and df = document frequency of each term.

Similarity: L2-normalized TF-IDF vectors; cosine similarity computed as a dot product (matrix–vector multiplication for efficiency).

Index persistence: Index serialized to `data/index.pkl` via Python's `pickle` module. Rebuild time on the test corpus: < 1 second.

4.3 Query Handling

The retriever accepts queries in any script – Devanagari, Latin transliteration, or English – and computes cosine similarity against all indexed chunks. The top-K chunks (default K=3) are returned ranked by score.

5. Generation Mechanism

5.1 LLM: TinyLlama 1.1B via Ollama

The generator uses [Ollama](#) to serve `tinylama` (TinyLlama 1.1B Chat) locally. Ollama performs pure CPU inference when no GPU is detected, satisfying the CPU-only requirement.

Model choice rationale:

- 1.1B parameters → fits comfortably in 4–6GB RAM
- GGUF quantized format → optimized for CPU inference
- Instruction-tuned variant → follows the RAG prompt template reliably
- No internet connectivity required after initial model download

5.2 Prompt Design

The RAG prompt follows a context-grounded instruction format:

```
You are a helpful assistant that answers questions based only on the provided Sanskrit text context.
If the answer is not present in the context, say "I could not find relevant information."
Do not make up information.
```

```
CONTEXT:
[retrieved chunks concatenated with source labels]
```

```
QUESTION:
[user query]
```

```
ANSWER:
```

Key design choices:

- **Grounding instruction:** Explicitly restricts the model to the provided context, reducing hallucination on low-resource Sanskrit content
- **Source labeling:** Each context chunk is labeled with its source file for traceability
- **Low temperature (0.1):** Favors factual, deterministic responses over creative generation

5.3 Fallback Behavior

If Ollama is unavailable (not installed or not running), the system gracefully degrades to a retrieval-only mode: the top-scoring chunk is returned directly as the response. This ensures the system remains functional even without the LLM component.

6. Performance Observations

All measurements taken on: Ryzen 5000 series CPU, 16GB RAM (CPU-only inference).

Metric	Observed Value
Ingestion time (26 chunks)	< 1 second
Index load time	< 50ms
Retrieval latency (top-3)	< 50ms
LLM generation time (TinyLlama)	30–90 seconds
Peak RAM usage (retrieval only)	~80MB
Peak RAM usage (with LLM)	~2.5GB

Retrieval accuracy (manual evaluation on 5 test queries):

Query	Top Chunk Relevant?	Score
■■■■■■■■■■■■■■■	☒	0.32
Who is Shankhanaad?	☒	0.41
What did the old woman do?	☒	0.38
■■■■■■■■■■	☒	0.29
bell in forest	☒	0.35

7. Known Limitations and Future Work

Limitations

1. **Script mismatch:** A query in Devanagari (कालिदास) and its transliteration (kAlIdaAsa) retrieve different chunks despite referring to the same entity. A transliteration normalization layer would unify these.
2. **LLM Sanskrit comprehension:** TinyLlama has minimal Sanskrit in its training data. Responses to Devanagari-heavy contexts are less coherent than responses to the English transliteration portions.
3. **Corpus size:** With only 26 chunks, retrieval is fast but coverage is limited. Performance scales linearly with corpus size.

Future Improvements

- **Multilingual embeddings:** Replace TF-IDF with a multilingual sentence-transformer (e.g., `paraphrase-multilingual-MiniLM-L12-v2`) once a Sanskrit-capable model becomes available
- **Transliteration normalization:** Map common IAST/Hunterian transliteration to canonical Devanagari before indexing
- **Hybrid retrieval:** Combine TF-IDF with BM25 for better handling of rare Sanskrit tokens
- **Larger LLM:** Use `llama3.2:3b` or `mistral:7b` via Ollama for better comprehension of mixed-script context

8. Conclusion

The implemented system demonstrates a complete, modular RAG pipeline for Sanskrit text that operates entirely on CPU hardware. The choice of TF-IDF retrieval over dense embeddings is linguistically motivated and produces reliable results on the mixed-script corpus. The Ollama-based generator provides a clean separation between retrieval and generation components while remaining reproducible on any standard laptop or desktop machine.